

8.

Introduction

SciPy (Scientific Python) is an open-source Python library built on top of NumPy for performing advanced scientific, mathematical, and engineering computations. It provides a large collection of modules for optimization, integration, interpolation, linear algebra, statistics, signal processing, image processing, and Fourier transforms. SciPy is widely used in Machine Learning, Data Science, scientific research, and engineering applications because of its efficient and reliable algorithms. It integrates seamlessly with other Python libraries such as NumPy, Pandas, Matplotlib, and Scikit-Learn. Due to its comprehensive collection of scientific functions, SciPy is considered one of the core libraries in the Python scientific computing ecosystem.

Import Library

```
from scipy import stats
```

Common Functions

1. **stats.describe()** – Returns descriptive statistics of data.
2. **stats.tmean()** – Calculates the trimmed mean.
3. **stats.mode()** – Finds the most frequent value.
4. **stats.median_abs_deviation()** – Calculates the median absolute deviation.
5. **stats.norm.pdf()** – Computes the probability density function.
6. **stats.norm.cdf()** – Computes the cumulative distribution function.
7. **stats.norm.rvs()** – Generates random samples from a normal distribution.
8. **stats.zscore()** – Computes Z-scores.
9. **stats.percentileofscore()** – Calculates the percentile rank.
10. **stats.rankdata()** – Assigns ranks to data values.
11. **linalg.det()** – Calculates the determinant of a matrix.
12. **linalg.inv()** – Computes the inverse of a matrix.
13. **linalg.eig()** – Computes eigenvalues and eigenvectors.
14. **linalg.solve()** – Solves linear equations.
15. **integrate.quad()** – Performs numerical integration.
16. **optimize.minimize()** – Finds the minimum of a function.
17. **fft.fft()** – Computes the Fast Fourier Transform.
18. **fft.ifft()** – Computes the Inverse Fourier Transform.
19. **interpolate.interp1d()** – Performs one-dimensional interpolation.
20. **signal.find_peaks()** – Finds peaks in a signal.
21. **signal.convolve()** – Performs convolution of signals.
22. **io.savemat()** – Saves data in MATLAB (.mat) format.
23. **io.loadmat()** – Loads MATLAB (.mat) files.
24. **special.factorial()** – Computes the factorial of numbers.
25. **special.comb()** – Computes combinations (nCr).

```
from scipy import stats, linalg, integrate, optimize, fft, interpolate, signal, io, special
import numpy as np

data = [2, 4, 4, 5, 6, 8, 9]

# 1. stats.describe()
print("1.", stats.describe(data))

# 2. stats.tmean()
print("2.", stats.tmean(data))

# 3. stats.mode()
print("3.", stats.mode(data))

# 4. stats.median_abs_deviation()
print("4.", stats.median_abs_deviation(data))

# 5. stats.norm.pdf()
print("5.", stats.norm.pdf(0))

# 6. stats.norm.cdf()
print("6.", stats.norm.cdf(0))
```

```

# 7. stats.norm.rvs()
print("7.", stats.norm.rvs(size=5))

# 8. stats.zscore()
print("8.", stats.zscore(data))

# 9. stats.percentileofscore()
print("9.", stats.percentileofscore(data, 5))

# 10. stats.rankdata()
print("10.", stats.rankdata(data))

# Matrix for linear algebra
A = np.array([[2,1],[5,3]])

# 11. linalg.det()
print("11.", linalg.det(A))

# 12. linalg.inv()
print("12.\n", linalg.inv(A))

# 13. linalg.eig()
values, vectors = linalg.eig(A)
print("13. Eigenvalues:", values)

# 14. linalg.solve()
b = np.array([3,7])
print("14.", linalg.solve(A,b))

# 15. integrate.quad()
result, error = integrate.quad(lambda x: x**2, 0, 2)
print("15.", result)

# 16. optimize.minimize()
res = optimize.minimize(lambda x: x**2 + 4*x + 4, x0=0)
print("16.", res.x)

# 17. fft.fft()
arr = np.array([1,2,3,4])
print("17.", fft.fft(arr))

# 18. fft.ifft()
print("18.", fft.ifft(fft.fft(arr)))

# 19. interpolate.interp1d()
x = np.array([0,1,2,3])
y = np.array([0,2,4,6])
f = interpolate.interp1d(x,y)
print("19.", f(1.5))

# 20. signal.find_peaks()
peaks, _ = signal.find_peaks([1,3,2,5,1,4])
print("20.", peaks)

# 21. signal.convolve()
print("21.", signal.convolve([1,2,3],[1,1]))

# 22. io.savemat()
io.savemat("sample.mat", {"array": arr})
print("22. sample.mat file saved")

# 23. io.loadmat()
loaded = io.loadmat("sample.mat")
print("23.", loaded["array"])

# 24. special.factorial()
print("24.", special.factorial(5))

# 25. special.comb()
print("25.", special.comb(5,2))

```

```

1. DescribeResult(nobs=np.int64(7), minmax=(np.int64(2), np.int64(9)), mean=np.float64(5.428571428571429), variance=np.float
2. 5.428571428571429
3. ModeResult(mode=np.int64(4), count=np.int64(2))
4. 1.0
5. 0.3989422804014327
6. 0.5
7. [ 0.68283991 -0.59186614  0.08039331  1.19083703 -0.30557521]
8. [-1.51789328 -0.63245553 -0.63245553 -0.18973666  0.25298221  1.13841996
   1.58113883]
9. 57.142857142857146
10. [1.  2.5 2.5 4.  5.  6.  7. ]
11. 1.0

```

```
12.
[[ 3. -1.]
 [-5.  2.]]
13. Eigenvalues: [0.20871215+0.j 4.79128785+0.j]
14. [ 2. -1.]
15. 2.6666666666666667
16. [-2.00000002]
17. [10.-0.j -2.+2.j -2.-0.j -2.-2.j]
18. [1.+0.j 2.+0.j 3.-0.j 4.+0.j]
19. 3.0
20. [1 3]
21. [1 3 5 3]
22. sample.mat file saved
23. [[1 2 3 4]]
24. 120.0
25. 10.0
```